NIVEL: SEMANA 02 DE PROGRAMACIÓN

Taller Práctico Grupal: TypeScript, Promesas Fake y Git en Equipo

Desafío colaborativo (2.5h): archivo único modularizado por secciones, interfaces anidadas, promesas feik con latencia/fallos y resolución guiada de conflictos en Git.

ORGANIZACIÓN	DURACIÓN TOTAL	ESTRUCTURA DE CÓDIGO	DINÁMICA GIT
Equipos de 4 Estudiantes	**2 Horas y 30 Minutos**	**index.ts (4 Secciones sin import)**	**4 Ramas + Conflictos + PR Review**

1. Dinámica de Trabajo y Arquitectura en Archivo Único

Dado que aún no se han visto módulos (`import` / `export`), todo el código vivirá en un solo archivo `index.ts` dividido en **4 secciones delimitadas por comentarios**. Cada estudiante trabaja en su propia rama tocando exclusivamente su sección asignada. Al hacer los Pull Requests hacia `main`, surgirán conflictos naturales de líneas adyacentes que el equipo resolverá en conjunto en VS Code.

Reglas del Sprint y Flujo de Git

1. **Estructura Base:** Crean la plantilla en `main` con los comentarios `// SECCIÓN 1` a `// SECCIÓN 4`.
2. **4 Ramas Simultáneas:** `feature/models-data`, `feature/async-api`, `feature/business-logic`, `feature/main-runner`.
3. **Simulación de Red:** La promesa simula latencia (2s) y un 20% de probabilidad de fallo (`reject`) para obligar el uso de `try/catch`.
4. **PR Review y Merge:** Cada PR requiere aprobación de al menos 2 compañeros y resolución asistida de conflictos.

2. Desglose de Responsabilidades Técnicas (4 Estudiantes)

Estudiante / Sección	Misión en index.ts	Retos Técnicos de Semana 2
Dev 1 (Sección 1: Modelos & Fixtures)	Escribe al inicio de `index.ts` al menos 2 interfaces con relaciones/sub-objetos (ej. `Producto` y `Proveedor`) y un arreglo feik con 8 registros con variedad de datos.	Sub-objetos tipados, tipos unión (ej. `'urgente' | 'normal'`), no usar any, tipos primitivos correctos.
Dev 2 (Sección 2: Promesa Feik con Latencia)	Crea la función asíncrona que retorna `Promise<T[]>` con `setTimeout` (2s). Con `Math.random() < 0.2` lanza `reject(new Error(...))` para simular caída de servidor.	Tipado genérico de promesas, encapsular callbacks con `setTimeout` y manejo de errores con `reject`.
Dev 3 (Sección 3: Lógica de Negocio y Métricas)	Implementa funciones de filtrado multi-condicional (&&, ||), cálculos acumulados (totales, impuestos, descuentos o promedios) y búsqueda por ID.	Bucles `for...of` o métodos de array (`filter`, `map`), validaciones condicionales y fórmulas matemáticas.
Dev 4 (Sección 4: Orquestador CLI y Fallbacks)	Crea la función `main()` con `async/await`. Consume la promesa de Dev 2, maneja errores con `try/catch` (mostrando alerta amigable) e imprime reporte formateado en consola.	Control asíncrono con `await`, captura de excepciones, template strings (`` `...` ``) y formato limpio en terminal.

3. Catálogo de Proyectos: Opciones 1 y 2 (Retail y Finanzas)

Proyectos con objetos anidados, cálculo de métricas financieras y tolerancia a caídas simuladas de conexión.

PROYECTO 01 Sistema de Inventario Retail con Tasas de Impuesto y Proveedor

E-Commerce & Logística

Caso Real: Una cadena minorista recibe stock de múltiples distribuidores. La app debe descargar el catálogo de la nube, calcular el IVA (13%) y descuento según volumen, alertar por productos críticos y manejar caídas de red del servidor central.

Rama de Git / Dev	Requerimientos Técnicos en TypeScript
`feature/inventory-models` **Dev 1**	Modelar `interface Proveedor { id: string; razonSocial: string; pais: string }` e `interface Producto { id: number; nombre: string; precioUnitario: number; categoria: 'tecnologia' | 'hogar' | 'ropa'; stock: number; proveedor: Proveedor }`. Armar array feik de 8 productos con al menos 2 con stock menor a 5 unidades.
`feature/inventory-api` **Dev 2**	Exportar `fetchInventario(): Promise<Producto[]>` con demora de 2.2s. Añadir lógica de simulación: si `Math.random() < 0.2`, rechazar con `Error("Servidor de inventarios no responde (503)")`.
`feature/inventory-analytics` **Dev 3**	Desarrollar: 1) `calcularValorTotal(lista: Producto[]): number` (suma total de stock × precio con 13% de impuesto incluido). 2) `alertarStockCritico(lista: Producto[], limite: number): Producto[]`. 3) `filtrarPorPaisProveedor(lista: Producto[], pais: string): Producto[]`.
`feature/cli-dashboard` **Dev 4**	Consumir la API con `async/await`. Si falla en el `catch`, reintentar una segunda vez. Al completar, imprimir resumen en consola: total monetario en almacén, desglose por categoría y tabla de alertas de reabastecimiento urgente.

PROYECTO 02 Plataforma de Microcréditos y Evaluación de Deuda

Fintech & Banca

Caso Real: Una fintech procesa solicitudes de préstamo. La consulta al buró de crédito es asíncrona; el sistema debe calificar el riesgo del cliente, calcular cuotas con interés simple y emitir veredictos de aprobación.

Rama de Git / Dev	Requerimientos Técnicos en TypeScript
`feature/credit-types` **Dev 1**	Definir `interface Solicitante { ci: string; nombre: string; ingresoMensual: number; scoreBancario: number }` y `interface SolicitudCredito { id: string; solicitante: Solicitante; montoPedido: number; plazoMeses: number; estado: 'pendiente' | 'aprobado' | 'rechazado' }` con 8 registros feik variados.
`feature/buro-service` **Dev 2**	Crear `obtenerSolicitudesBancarias(): Promise<SolicitudCredito[]>` con latencia de 2.5s. Añadir log de progreso (*"Conectando a central de riesgos..."*) y probabilidad del 20% de fallo con mensaje de tiempo de espera agotado (timeout).
`feature/scoring-rules` **Dev 3**	Crear función `evaluarRiesgo(solicitud: SolicitudCredito): boolean`: aprueba solo si el score es >= 650 y si la cuota mensual (monto + 10% interés anual / meses) no supera el 35% del `ingresoMensual`. Crear función de resumen total de dinero solicitado.
`feature/credit-console` **Dev 4**	Ejecutar el flujo completo en `main.ts` con `try/catch`. Mostrar lista detallada de evaluados con etiquetas `[APROBADO]` o `[RECHAZADO]`, justificando la causa en caso de rechazo, más el monto total colocado.

3. Catálogo de Proyectos: Opciones 3 y 4 (Logística y Salud)

Proyectos enfocados en tracking en tiempo real, priorización por criticidad médica y tarifas dinámicas.

PROYECTO 03 Sistema de Envíos Urbanos con Tarifa Dinámica y Paquetes Frágiles

Logística & Delivery

Caso Real: Una empresa de mensajería procesa despachos pendientes. La descarga de envíos tiene delay de red; se deben calcular recargos por peso y distancia, asegurar seguro para objetos frágiles y despachar la ruta más óptima.

Rama de Git / Dev	Requerimientos Técnicos en TypeScript
`feature/shipping-types` **Dev 1**	Modelar `interface Destinatario { nombre: string; direccion: string; telefono: string }` y `interface Paquete { trackingId: string; pesoKg: number; distanciaKm: number; esFragil: boolean; express: boolean; destinatario: Destinatario }`. Crear array feik con 8 paquetes heterogéneos.
`feature/dispatch-api` **Dev 2**	Crear `descargarManifiesto(): Promise<Paquete[]>` con demora de 2s. Implementar el 20% de probabilidad de error (*"Fallo en sincronización GPS/Almacén"*) retornando un rechazo tipado.
`feature/tariff-calculator` **Dev 3**	Implementar `calcularCostoEnvio(p: Paquete): number`: Tarifa base $15 + ($2.5 × km) + ($1.5 × kg). Si es frágil sumar seguro de $20; si es express multiplicar total por 1.25. Crear función adicional para ordenar paquetes de menor a mayor distancia.
`feature/dispatch-runner` **Dev 4**	Consumir la promesa en `main.ts` con `try/catch`. Si hay error, cargar un array de respaldo local. Imprimir itinerario de chofer con subtotales, costo total a cobrar del camión y paquetes prioritarios destacados.

PROYECTO 04 Triage Hospitalario de Emergencias con Signos Vitales

Salud & Emergencias

Caso Real: Un centro médico recibe la lista de pacientes en guardia. La descarga del servidor clínico simula latencia; el sistema debe evaluar signos vitales, asignar prioridad y alertar casos de riesgo vital.

Rama de Git / Dev	Requerimientos Técnicos en TypeScript
`feature/triage-models` **Dev 1**	Definir `interface SignosVitales { presionSistolica: number; pulsoBpm: number; oxigenoSpO2: number }` y `interface PacienteIngreso { historialId: string; nombre: string; edad: number; sintomas: string[]; signos: SignosVitales; nivelPrioridad?: 1 | 2 | 3 }`. Crear 8 pacientes feik.
`feature/clinical-service` **Dev 2**	Exportar `consultarAdmisiones(): Promise<PacienteIngreso[]>` con 2.3 segundos de espera. Emitir error si falla la comunicación con la base de emergencias (20% de probabilidad).
`feature/triage-engine` **Dev 3**	Crear función `determinarPrioridad(p: PacienteIngreso): 1 | 2 | 3`: Prioridad 1 (Rojo/Grave) si SpO2 < 90 o pulso > 120 o pulso < 50; Prioridad 2 (Amarillo) si edad > 65 o presión > 140; Prioridad 3 (Verde) casos estables. Función de conteo por nivel.
`feature/triage-monitor` **Dev 4**	Orquestar en `main.ts` con `async/await` y manejo de excepciones. Imprimir orden de atención empezando por los de Prioridad 1, advirtiendo con alarmas visuales en texto a los pacientes con hipoxemia grave.

3. Catálogo de Proyectos: Opciones 5 y 6 (Hotelería y Gaming)

PROYECTO 05 **Sistema de Reservas Hoteleras con Temporadas y Penalizaciones** — Turismo & Hotelería

Rama de Git / Dev	Requerimientos Técnicos en TypeScript
`feature/hotel-types` **Dev 1**	`interface Huesped { dni: string; nombreCompleto: string; esVip: boolean }` y `interface Reserva { id: string; huesped: Huesped; noches: number; tipoHabitacion: 'simple' | 'suite' | 'familiar'; costoBasePorNoche: number; estado: 'confirmada' | 'cancelada' }`. Array de 8 reservas feik.
`feature/booking-api` **Dev 2**	Función `sincronizarReservas(): Promise<Reserva[]>` con latencia de 2.2s y fallo aleatorio del 20% (*"Error 504 Gateway Timeout con Central Hotelera"*).
`feature/revenue-rules` **Dev 3**	Función `liquidarReserva(r: Reserva): number`: multiplica noches × costoBase. Si es VIP aplica 15% de descuento; si la habitación es suite añade cargo de servicio del 10%. Si está cancelada, cobra 40% de penalización sobre el total.
`feature/hotel-report` **Dev 4**	Consumir con `async/await` y capturar caídas. Imprimir balance financiero del día: ingresos confirmados vs dinero retenido por penalidades de cancelación y listado de suites ocupadas.

PROYECTO 06 **Plataforma de Torneo Esports con Estadísticas y Ranking ELO** — Gaming & Competitivo

Rama de Git / Dev	Requerimientos Técnicos en TypeScript
`feature/gamer-models` **Dev 1**	`interface Stats { partidasJugadas: number; victorias: number; kills: number; deaths: number }` y `interface Jugador { id: number; tag: string; rol: 'duelista' | 'soporte' | 'tanque'; eloInicial: number; stats: Stats }`. Array feik con 8 competidores.
`feature/matchmaking-api` **Dev 2**	Función `cargarTablaTorneo(): Promise<Jugador[]>` con retraso de 2s y probabilidad de fallo del 20% por "servidor de emparejamiento ocupado".
`feature/rankings-logic` **Dev 3**	Función `calcularKDA(j: Jugador): number` (kills / (deaths || 1)), cálculo de nuevo ELO sumando 25 pts por victoria y restando 15 por derrota, y función para ordenar jugadores por porcentaje de winrate.
`feature/leaderboard-ui` **Dev 4**	Implementar consumo asíncrono con control de fallos. Imprimir la tabla de posiciones oficial (Leaderboard) formateada con tags, KDA individual, ELO final y mención al MVP del torneo.

4. Distribución del Sprint de 2 Horas y Media (150 minutos)

Bloque	Acción de los 4 Integrantes	Evidencia a Revisar por el Docente
Min 0 – 20	Diseño de interfaces en borrador, creación de repo, ramas de cada Dev y estructura modular de archivos.	Ramas creadas en GitHub sin colisiones previas.
Min 20 – 85	Programación modular en paralelo: Dev 1 tipa datos; Dev 2 arma promesa con retardo y fallos; Dev 3 cálculos; Dev 4 runner CLI.	Tipado estricto (0% any), uso de `interface` y `reject()`.
Min 85 – 125	Apertura de 4 Pull Requests, revisión cruzada de código (Code Review), resolución de conflictos y merges a `main`.	Historial de PRs aprobados y mergeados limpiamente.
Min 125 – 150	Ejecución final en vivo ante el docente probando éxito y captura de errores simulados en consola.	Script ejecutándose con datos limpios y fallback funcional.